

# Содержание

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 1. Зачем нужен этот файл .....                                          | 2  |
| 2. Общая схема всей лабораторной .....                                  | 2  |
| 3. Где какие параметры применяются .....                                | 2  |
| 4. Pipeline данных .....                                                | 3  |
| 4.1. Шаг 1. Загрузка датасета .....                                     | 3  |
| 4.2. Шаг 2. Уменьшение выборки .....                                    | 3  |
| 4.3. Шаг 3. Split train/val .....                                       | 4  |
| 4.4. Шаг 4. DataLoader .....                                            | 4  |
| 5. Что приходит на вход CNN .....                                       | 4  |
| 6. Pipeline одного прохода через модель .....                           | 4  |
| 6.1. Общая схема forward-pass .....                                     | 5  |
| 6.2. Внутри ConvBlockA .....                                            | 5  |
| 6.3. Внутри ConvBlockB .....                                            | 5  |
| 7. Как меняются размеры тензора .....                                   | 5  |
| 7.1. Формула для свёртки .....                                          | 5  |
| 7.2. Формула для pooling .....                                          | 5  |
| 7.3. Пример размеров для одной популярной конфигурации .....            | 6  |
| 8. Как строится модель из списка channels .....                         | 6  |
| 8.1. Пример 1 .....                                                     | 6  |
| 8.2. Пример 2 .....                                                     | 7  |
| 9. Где в pipeline применяется каждый архитектурный параметр .....       | 7  |
| 10. Pipeline одной training-итерации .....                              | 7  |
| 10.1. Схема training-step .....                                         | 7  |
| 10.2. То же самое словами .....                                         | 8  |
| 11. Что именно происходит в <code>loss = criterion(out, y)</code> ..... | 8  |
| 12. Pipeline одной эпохи .....                                          | 8  |
| 13. Pipeline validation после эпохи .....                               | 8  |
| 14. Как работает early stopping .....                                   | 9  |
| 15. Pipeline перебора конфигураций в блоке 1 .....                      | 9  |
| 15.1. Общая схема grid search .....                                     | 9  |
| 15.2. Что значит «заново» .....                                         | 9  |
| 16. Какой результат сохраняется для каждой конфигурации .....           | 9  |
| 17. Финальное дообучение лучшей модели .....                            | 10 |
| 17.1. Схема .....                                                       | 10 |
| 18. Pipeline блока 2: оптимизаторы и переобучение .....                 | 10 |
| 18.1. Часть 1. Сравнение оптимизаторов .....                            | 10 |
| 18.2. Часть 2. Демонстрация переобучения .....                          | 11 |
| 19. Полный pipeline в одном списке .....                                | 11 |
| 20. Что особенно важно запомнить .....                                  | 11 |

# 1. Зачем нужен этот файл

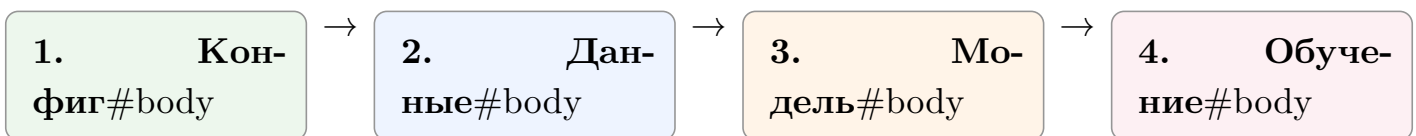
Этот документ нужен как **визуальная карта всей лабораторной**. Если `main_teor.typ` объясняет теорию, а `main_code.typ` объясняет код, то здесь цель другая: показать **pipeline целиком**.

То есть ответить на вопросы:

- откуда берутся данные;
- как из них строятся `train`, `val`, `test`;
- как батч проходит через CNN;
- какие размеры тензоров получаются после каждого шага;
- где применяются гиперпараметры;
- как считается `loss`;
- как работает `backward` и обновление весов;
- как выбирается лучшая модель.

## 2. Общая схема всей лабораторной

Ниже самая верхнеуровневая схема всей работы:



После этого идёт второй уровень логики:

- в **блоке 1** лабораторная много раз повторяет шаги 3 и 4 для разных конфигураций;
- в **блоке 2** фиксирует хорошую архитектуру и отдельно исследует оптимизаторы и переобучение.

## 3. Где какие параметры применяются

Очень полезно сразу видеть не просто список гиперпараметров, а **место их применения**.

| Параметр                     | Где применяется                                              | Что меняет                                                             |
|------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------|
| DATASET_NAME                 | <code>data.py</code><br><code>load_image_dataset(...)</code> | -> Какой датасет будет загружен: MNIST или FashionMNIST.               |
| TRAIN_SUBSET,<br>TEST_SUBSET | <code>block1_main.py</code> ,<br><code>block2_main.py</code> | Размер подвыборок для ускорения экспериментов.                         |
| VAL_RATIO                    | <code>data.py</code><br>> <code>split_train_val(...)</code>  | - Какую долю <code>train</code> выделить под <code>validation</code> . |
| BATCH_SIZE                   | <code>make_loaders(...)</code> ,<br><code>DataLoader</code>  | Сколько изображений идёт в одном батче.                                |
| block_type                   | <code>cnn.py</code> -> ImageCNN                              | Использовать блок А или В.                                             |

|                     |                                   |                                                     |
|---------------------|-----------------------------------|-----------------------------------------------------|
| channels            | cnn.py -> цикл построения слоёв   | Сколько выходных каналов у каждого блока.           |
| conv_k, conv_s      | Conv2d                            | Размер ядра и шаг свёртки.                          |
| pool_k, pool_s      | MaxPool2d                         | Размер окна pooling и его шаг.                      |
| dropout_rate        | ImageCNN -> classifier            | Сила dropout перед финальным линейным слоем.        |
| lr                  | make_optimizer(...)               | Скорость изменения весов на каждом шаге.            |
| PATIENCE, MIN_DELTA | train_model(...)                  | Когда остановить обучение без улучшения validation. |
| TARGET_ACC          | train_model(...), сводные графики | Контрольная цель качества, например 0.90.           |

## 4. Pipeline данных

Сначала разберём именно путь данных до подачи в сеть.

### 4.1. Шаг 1. Загрузка датасета

В data.py вызывается:

```
train, test, info = load_image_dataset(name, data_root)
```

На этом шаге происходит:

1. выбор датасета по имени;
2. добавление ToTensor();
3. при необходимости добавление Normalize(mean, std);
4. загрузка обучающей и тестовой части.

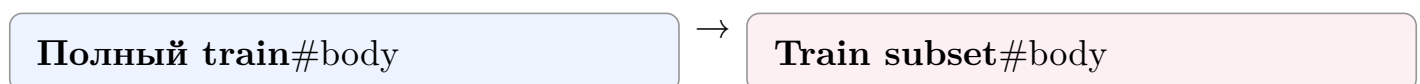
Схема:



### 4.2. Шаг 2. Уменьшение выборки

Чтобы сетка экспериментов не была слишком тяжёлой, используется take\_subset(...).

Схема:



### 4.3. Шаг 3. Split train/val

Из train-подвыборки строятся две части:

- `train_ds` — для обновления весов;
- `val_ds` — для контроля качества и выбора модели.

Схема:



### 4.4. Шаг 4. DataLoader

После этого из датасетов создаются `DataLoader`.

Смысл:

- `train_loader` выдаёт батчи и перемешивает `train`;
- `val_loader` и `test_loader` выдают батчи без перемешивания.

Схема:



## 5. Что приходит на вход CNN

Для MNIST или FashionMNIST одно изображение имеет форму:

$(1, 28, 28)$ .

Здесь:

- 1 — число каналов, потому что изображение серое;
- 28 x 28 — высота и ширина.

Если батч размера 128, то на вход модели приходит тензор формы:

$(128, 1, 28, 28)$ .

Это один из самых важных моментов всей темы. Нужно чётко понимать: модель почти никогда не видит один объект, она обычно видит **батч объектов**.

## 6. Pipeline одного прохода через модель

Теперь покажем, как один батч проходит через CNN.

## 6.1. Общая схема forward-pass



## 6.2. Внутри ConvBlockA

Блок А в коде устроен так:

```
Conv2d -> ReLU -> MaxPool2d
```

Схема:

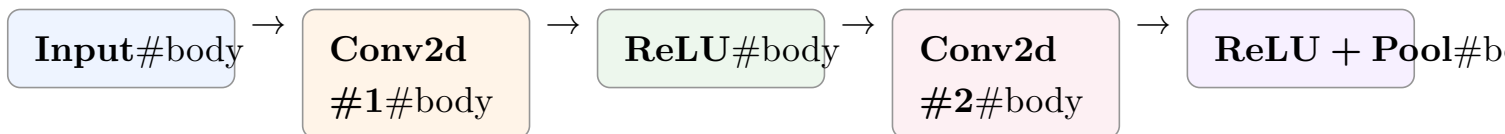


## 6.3. Внутри ConvBlockB

Блок В глубже:

```
Conv2d -> ReLU -> Conv2d -> ReLU -> MaxPool2d
```

Схема:



Разница в смысле такая:

- А извлекает признаки проще и быстрее;
- В извлекает признаки глубже, потому что внутри блока две свёртки.

## 7. Как меняются размеры тензора

Это одна из самых важных частей для понимания CNN. Нужно видеть не только «какой слой стоит после какого», но и **какой размер получается после него**.

### 7.1. Формула для свёртки

Если входной размер по одной координате равен  $in$ , а параметры свёртки равны  $kernel = k$ ,  $stride = s$ ,  $padding = p$ , то выходной размер:

$$out = \left\lfloor \frac{in + 2p - k}{s} + 1 \right\rfloor.$$

### 7.2. Формула для pooling

Для pooling используется та же идея:

$$\text{out} = \left\lfloor \frac{\text{in} - k}{s} + 1 \right\rfloor.$$

В лабораторной наиболее типичный случай такой:

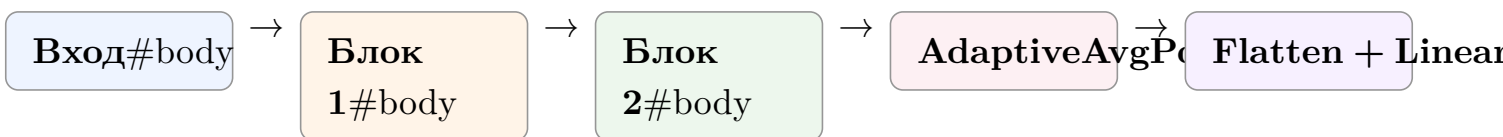
- conv\_k = 3;
- conv\_s = 1;
- padding = 1;
- pool\_k = 2;
- pool\_s = 2.

### 7.3. Пример размеров для одной популярной конфигурации

Возьмём конфиг:

```
block_type = 'b'
channels = (32, 64)
conv_k = 3
conv_s = 1
pool_k = 2
pool_s = 2
```

Тогда pipeline размеров выглядит так:



Что здесь произошло:

- первая свёртка с k=3, s=1, p=1 сохраняет размер 28 x 28;
- pooling 2 x 2 со stride 2 уменьшает размер до 14 x 14;
- второй блок делает то же самое ещё раз и получаем 7 x 7;
- затем каждый из 64 каналов усредняется до одной точки;
- остаётся вектор длины 64 на каждый объект;
- линейный слой переводит эти 64 признака в 10 логитов классов.

## 8. Как строится модель из списка channels

Очень важно видеть, как параметр channels реально разворачивается в архитектуру.

### 8.1. Пример 1

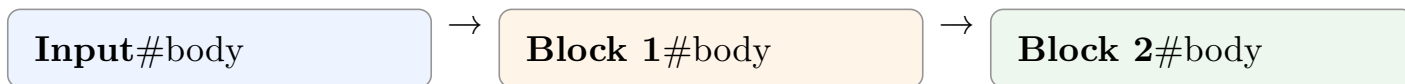
```
channels = (16, 32)
```

Это означает:

- первый блок получает in\_channels = 1 и выдаёт 16 каналов;

- второй блок получает уже 16 каналов и выдаёт 32.

Схема:



## 8.2. Пример 2

```
channels = (16, 32, 64)
```

Схема:



То есть длина `channels` фактически задаёт **глубину модели**, а сами значения задают **ширину** по каналам.

## 9. Где в pipeline применяется каждый архитектурный параметр

### Параметр

`block_type`

### Как он проявляется в pipeline

Выбирает схему А или В, то есть сколько свёрток внутри одного блока.

`channels`

Определяет число выходных каналов после каждого блока.

`conv_k`

Меняет размер окна, через которое фильтр «смотрит» на изображение.

`conv_s`

Меняет скорость перемещения фильтра и влияет на размер выходной карты признаков.

`pool_k`

Определяет размер окна pooling.

`pool_s`

Определяет, насколько быстро pooling уменьшает размерность.

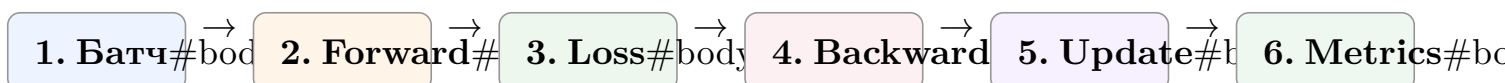
`dropout_rate`

Определяет, какая доля признаков будет занулена перед финальным классификатором.

## 10. Pipeline одной training-итерации

Теперь разберём не только прохождение через слои, но и то, что происходит на **одном обучающем шаге**.

### 10.1. Схема training-step



## 10.2. То же самое словами

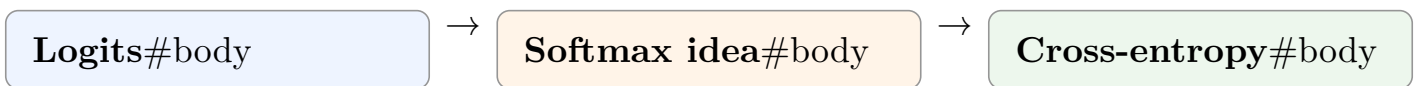
1. Берём батч изображений и истинных меток.
2. Прогоняем изображения через модель.
3. Получаем логиты классов.
4. Сравниваем логиты с истинными метками через `CrossEntropyLoss`.
5. Считаем градиенты по всем параметрам.
6. Оптимизатор обновляет параметры.

Это и есть элементарный акт обучения нейросети.

## 11. Что именно происходит в `loss = criterion(out, y)`

В лабораторной используется `CrossEntropyLoss`.

Схема вычисления по смыслу такая:



Пример по одному объекту:

- если правильный класс получил вероятность `0.95`, `loss` маленький;
- если правильный класс получил вероятность `0.10`, `loss` большой.

## 12. Pipeline одной эпохи

Одна эпоха — это не один шаг, а **полный проход по всем батчам `train-loader`**.

Схема:

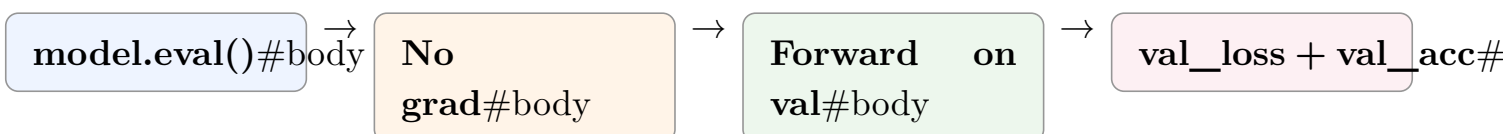


После завершения `train`-части эпохи идёт `validation`.

## 13. Pipeline validation после эпохи

После каждой эпохи модель проверяется на `val_loader`.

Схема:



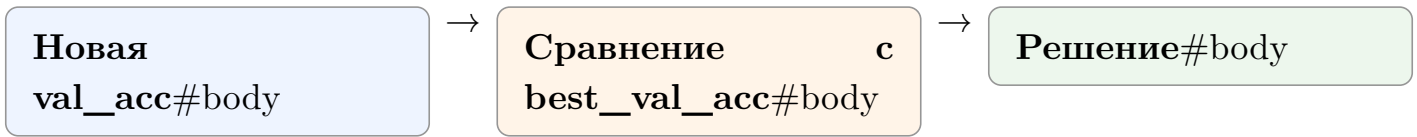
Это важный шаг, потому что именно здесь видно, учит ли модель общую закономерность, а не только запоминает `train`.



## 14. Как работает early stopping

В `train_model(...)` после каждой эпохи сравнивается новая `val_acc` с лучшей предыдущей.

Схема логики:



Дальше:

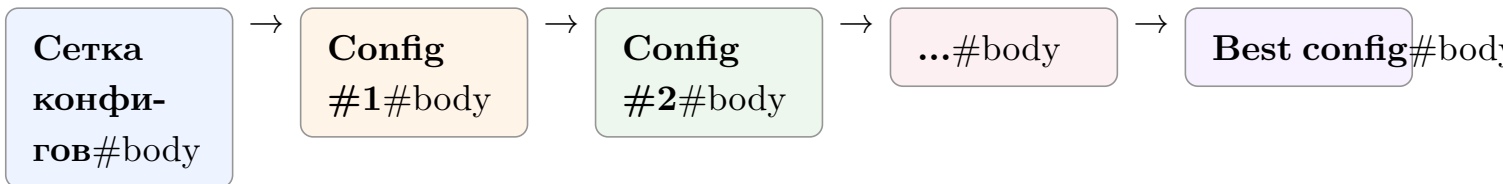
- если `bad < PATIENCE`, обучение продолжается;
- если `bad >= PATIENCE`, обучение останавливается.

То есть early stopping — это защитный фильтр от бесконечного обучения без смысла и от переобучения.

## 15. Pipeline перебора конфигураций в блоке 1

Теперь посмотрим не на одну модель, а на **весь цикл поиска лучшей архитектуры**.

### 15.1. Общая схема grid search



### 15.2. Что значит «заново»

Для каждой конфигурации происходит полный отдельный pipeline:

- создаётся новая модель;
- инициализируются новые веса;
- создаётся новый оптимизатор;
- модель обучается с нуля;
- сохраняются её история и метрики.

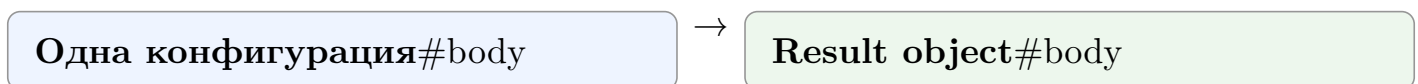
Это нужно для честного сравнения архитектур.

## 16. Какой результат сохраняется для каждой конфигурации

После обучения одного конфига сохраняется словарь результата, где есть:

- history;
- best\_val\_acc;
- best\_epoch;
- epoch\_to\_target;
- duration\_sec;
- trained\_epochs;
- cfg;
- model;
- n\_params.

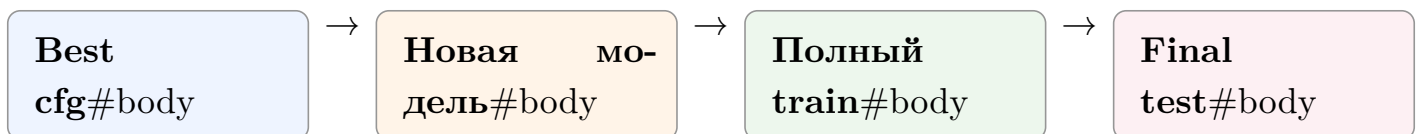
Схема:



## 17. Финальное дообучение лучшей модели

После выбора лучшего конфига происходит ещё один отдельный pipeline.

### 17.1. Схема



Почему это правильно:

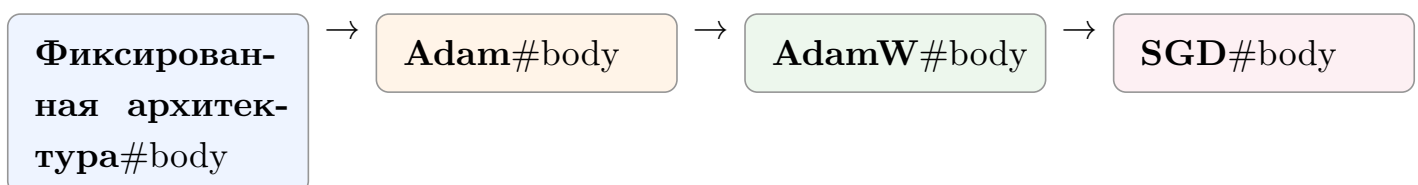
- validation уже выполнила свою роль при выборе модели;
- теперь можно использовать все train-данные для максимального качества;
- итоговая оценка делается по test.

## 18. Pipeline блока 2: оптимизаторы и переобучение

Во втором блоке pipeline немного другой.

### 18.1. Часть 1. Сравнение оптимизаторов

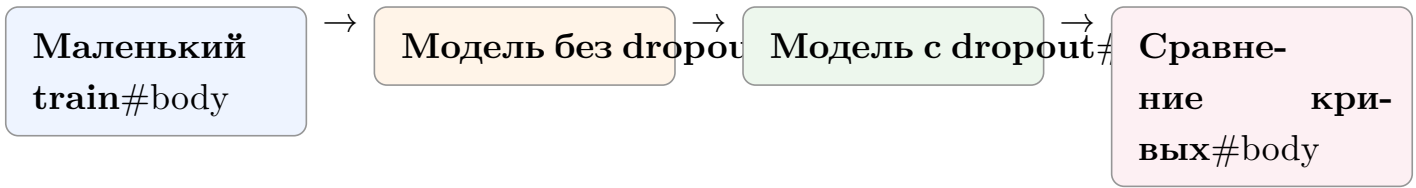
Схема:



Идея здесь такая: архитектура фиксирована, меняется только способ обновления весов.

## 18.2. Часть 2. Демонстрация переобучения

Схема:



## 19. Полный pipeline в одном списке

Если сжать всю лабораторную в одну последовательность, получится такой маршрут.

1. Задать конфигурацию запуска.
2. Загрузить датасет и подготовить transforms.
3. Получить `train`, `val`, `test`.
4. Построить `DataLoader` для батчей.
5. Построить одну CNN-конфигурацию.
6. Прогонять батчи через `forward`.
7. Считать `CrossEntropyLoss`.
8. Делать `backward` и `optimizer.step()`.
9. После каждой эпохи считать validation-метрики.
10. Хранить лучшую модель по validation.
11. Повторить это для всех конфигураций в сетке.
12. Выбрать лучшую конфигурацию.
13. Переобучить её на полном `train`.
14. Посчитать итоговую `test accuracy`.
15. Построить графики и `confusion matrix`.

## 20. Что особенно важно запомнить

Если тебе нужно быстро воспроизвести pipeline по памяти, то запомни пять ключевых уровней.

1. **Данные:** `dataset -> subset -> split -> dataloader`.
2. **Архитектура:** `input -> conv blocks -> global pooling -> linear`.
3. **Один шаг обучения:** `forward -> loss -> backward -> optimizer.step()`.
4. **Одна эпоха:** `train all batches -> validate`.
5. **Вся лабораторная:** `grid search -> best config -> final retrain -> test`.

Если эта пятиуровневая схема ясна, то и код, и теория этой лабораторной уже собираются в цельную картину.